

Yulu Hypothesis Testing

Section 1 - Business Understanding

Yulu is India's leading micro-mobility service provider offering shared electric bicycles for daily commuting. Recently, the company has experienced a decline in revenue and wants to identify the factors influencing customer demand for its rental services.

Objective of the Analysis

The objective of this analysis is to examine the relationship between bike rental demand and various environmental and operational factors such as weather conditions, season, temperature, humidity, and working days.

Statistical hypothesis testing will be used to determine whether these factors significantly affect the number of electric bicycles rented.

Section 2 - Importing libraries and loading dataset

```
# Step 1 Importing Libraries

# Data Manipulation
import pandas as pd
import numpy as np

# Data Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Statistical Analysis
from scipy import stats
from scipy.stats import ttest_ind, f_oneway, chi2_contingency
from scipy.stats import shapiro, levene

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

# Plot aesthetics
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (8,5)
```

```
# Step 2 Loading the Dataset

!gdown 1QmWJDIX9_G4RuF547MRwvyzPpB6bibX

df = pd.read_csv("yulu_csv.csv")

# Display first five rows
df.head()
```

Downloading...
 From: https://drive.google.com/uc?id=1QmWJDIX9_G4RuF547MRwvyzPpB6bibX
 To: /content/yulu_csv.csv
 100% 659k/659k [00:00<00:00, 76.6MB/s]

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Step 3 Basic Exploration
```

```
# Shape of the dataset
```

```
print(f"Number of rows: {df.shape[0]}")
print(f"Number of columns: {df.shape[1]}")
print(f"Column names: {df.columns.tolist()}")
```

```
Number of rows: 10886
```

```
Number of columns: 12
```

```
Column names: ['datetime', 'season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'casual',
```

```
# Statistical summary
```

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max	
season	10886.0	2.506614	1.116174	1.00	2.0000	3.000	4.0000	4.0000	
holiday	10886.0	0.028569	0.166599	0.00	0.0000	0.000	0.0000	1.0000	
workingday	10886.0	0.680875	0.466159	0.00	0.0000	1.000	1.0000	1.0000	
weather	10886.0	1.418427	0.633839	1.00	1.0000	1.000	2.0000	4.0000	
temp	10886.0	20.230860	7.791590	0.82	13.9400	20.500	26.2400	41.0000	
atemp	10886.0	23.655084	8.474601	0.76	16.6650	24.240	31.0600	45.4550	
humidity	10886.0	61.886460	19.245033	0.00	47.0000	62.000	77.0000	100.0000	
windspeed	10886.0	12.799395	8.164537	0.00	7.0015	12.998	16.9979	56.9969	
casual	10886.0	36.021955	49.960477	0.00	4.0000	17.000	49.0000	367.0000	
registered	10886.0	155.552177	151.039033	0.00	36.0000	118.000	222.0000	886.0000	
count	10886.0	191.574132	181.144454	1.00	42.0000	145.000	284.0000	977.0000	

```
# Step 4 Missing values
```

```
df.isnull().sum()
```

```

datetime    0
season      0
holiday     0
workingday  0
weather     0
temp        0
atemp       0
humidity    0
windspeed   0
casual      0
registered  0
count       0
dtype: int64
```

Interpretation: This makes our job easier since there are NO missing values

```
# Step 5 Duplicate records
```

```
print(f"Number of duplicate records: {df.duplicated().sum()}")
```

```
Number of duplicate records: 0
```

```
# Step 6 Converting Categorical Variables
```

```

categorical_cols = [
    "season",
    "holiday",
    "workingday",

```

```

    "weather"
]

for cols in categorical_cols:
    df[cols] = df[cols].astype("category")

```

Step 7 Convert Datetime

```
df['datetime'] = pd.to_datetime(df['datetime'])
```

Step 8 Feature Engineering

Extract useful information from the datetime column

```

df['year'] = df['datetime'].dt.year
df['month'] = df['datetime'].dt.month
df['day'] = df['datetime'].dt.day
df['hour'] = df['datetime'].dt.hour
df['day_name'] = df['datetime'].dt.day_name()

```

Step 9 Check unique values

```

for col in categorical_cols:
    print(f"\n{col}")
    print(df[col].value_counts())

```

```

season
season
4    2734
2    2733
3    2733
1    2686
Name: count, dtype: int64

```

```

holiday
holiday
0    10575
1      311
Name: count, dtype: int64

```

```

workingday
workingday
1    7412
0    3474
Name: count, dtype: int64

```

```

weather
weather
1    7192
2    2834
3     859
4         1
Name: count, dtype: int64

```

Step 10 Final data check

This is a quick hygiene check before moving into visual analysis

```
df.head()
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count	year	month
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16	2011	1
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40	2011	1
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32	2011	1
3	2011-01-01	1	0	0	1	9.84	14.395	75	0.0	3	10	13	2011	1

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Section 3 - Exploratory Data Analysis

3.1 Univariate Analysis

```
# Continuous Variables

# We'll analyze the following continuous variables

continuous_cols = [
    'temp',
    'atemp',
    'humidity',
    'windspeed'
]

# Summary Statistics

df[continuous_cols].describe().T
```

	count	mean	std	min	25%	50%	75%	max	
temp	10886.0	20.230860	7.791590	0.82	13.9400	20.500	26.2400	41.0000	
atemp	10886.0	23.655084	8.474601	0.76	16.6650	24.240	31.0600	45.4550	
humidity	10886.0	61.886460	19.245033	0.00	47.0000	62.000	77.0000	100.0000	
windspeed	10886.0	12.799395	8.164537	0.00	7.0015	12.998	16.9979	56.9969	

```
# Distribution plots

for col in continuous_cols:
    plt.figure(figsize=(8,4))
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.show()
```

[Show hidden output](#)

```
# Boxplots

for col in continuous_cols:
    plt.figure(figsize=(7,2.5))
    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')
    plt.show()
```

[Show hidden output](#)

```
# Countplots

for col in categorical_cols:
    plt.figure(figsize=(7,4))
    sns.countplot(data=df, x=col)
    plt.title(f'Count Plot of {col}')
    plt.show()
```

[Show hidden output](#)

Key Observations

- Variables such as temp and atemp exhibit approximately symmetric distributions
- Boxplots reveal the presence of outliers in variables such as windspeed; however, these likely represent natural variations rather than data quality issues
- Most observations correspond to non-holidays, working days, and clear weather conditions
- The observations are reasonably distributed across the four seasons, making seasonal comparisons meaningful

3.2 Bivariate Analysis

```
# Working Day vs Bike Rentals

plt.figure(figsize=(6,4))

sns.boxplot(data=df,
            x='workingday',
            y='count')
```

```
plt.title('Working Day vs Bike Rentals')

plt.show()
```

[Show hidden output](#)

```
# Season vs Bike Rentals

plt.figure(figsize=(6,4))

sns.boxplot(data=df,
             x='season',
             y='count')

plt.title('Season vs Bike Rentals')

plt.show()
```

[Show hidden output](#)

```
# Weather vs Bike Rentals

plt.figure(figsize=(7,4))

sns.boxplot(data=df,
             x='weather',
             y='count')

plt.title('Weather vs Bike Rentals')

plt.show()
```

[Show hidden output](#)

```
# Temperature vs Bike Rentals

plt.figure(figsize=(7,4))

sns.scatterplot(data=df,
                 x='temp',
                 y='count')

plt.title('Temperature vs Bike Rentals')

plt.show()
```

[Show hidden output](#)

```
# Humidity vs Bike Rentals

plt.figure(figsize=(7,4))

sns.scatterplot(data=df,
                 x='humidity',
                 y='count')

plt.title('Humidity vs Bike Rentals')

plt.show()
```

[Show hidden output](#)

```
# Correlation Heatmap

numeric_df = df.select_dtypes(include=np.number) # it's better to include only numeric variables

plt.figure(figsize=(10,8))
sns.heatmap(numeric_df.corr(),
            annot=True,
            cmap='coolwarm',
            fmt='.2f')

plt.title('Correlation Heatmap')
plt.show()
```

[Show hidden output](#)

Key Observations

- Rental demand appears higher on working days than on non-working days, suggesting that commuting may contribute significantly to bike usage
- The distribution of rentals varies across seasons, indicating a potential seasonal influence on demand
- Bike rentals tend to decrease as weather conditions become more severe, highlighting the impact of weather on customer behavior
- Temperature shows a positive relationship with bike rentals, whereas humidity exhibits a weaker negative relationship
- The correlation heatmap indicates a strong positive correlation between temp and atemp, while registered users contribute more strongly to total rentals than casual users

EDA Summary

The exploratory analysis indicates that environmental and operational factors such as working days, seasons, weather conditions, and temperature are associated with variations in bike rental demand. These visual patterns suggest that several independent variables may significantly influence the number of bike rentals.

However, visual analysis alone is insufficient to establish statistical significance. Therefore, formal hypothesis testing will be conducted in the next section to validate whether the observed differences are statistically significant.

Section 4 - Hypothesis Testing

This section directly answers Yulu's business questions using statistical tests.

4.1 Two-Sample T-Test

Objective: To determine whether the average number of electric cycles rented differs between working days and non-working days.

Step 1: Define the Hypothesis

Null Hypothesis

The mean number of bike rentals is the same on working and non-working days.

Alternate Hypothesis

The mean number of bike rentals differs between working and non-working days.

Step 2: Create the Two Groups

```
working_day = df[df['workingday'] == 1]['count']
non_working_day = df[df['workingday'] == 0]['count']
```

Step 3: Check Assumption 1 — Normality

```
fig, axes = plt.subplots(1, 2, figsize=(12,5))

sns.histplot(working_day, kde=True, ax=axes[0])
axes[0].set_title("Working Day")

sns.histplot(non_working_day, kde=True, ax=axes[1])
axes[1].set_title("Non-Working Day")

plt.tight_layout()
plt.show()
```

[Show hidden output](#)

Shapiro Wilk Test

```
from scipy.stats import shapiro

print("Working Day:", shapiro(working_day.sample(3000, random_state=42)))
print("Non Working Day:", shapiro(non_working_day.sample(3000, random_state=42)))

Working Day: ShapiroResult(statistic=np.float64(0.8778080225417806), pvalue=np.float64(1.4641284740766062e-43))
Non Working Day: ShapiroResult(statistic=np.float64(0.88596415903702), pvalue=np.float64(1.7056099317357474e-42))
```

Step 4: Check Assumption 2 — Equal Variance

```
from scipy.stats import levene

stat, p = levene(working_day, non_working_day)

print("Levene Statistic :", stat)
print("p-value :", p)
```

```
Levene Statistic : 0.004972848886504472
p-value : 0.9437823280916695
```

Step 5: Perform Independent T-Test

```
equal_var = p > 0.05

t_stat, p_value = stats.ttest_ind(
    working_day,
    non_working_day,
    equal_var=equal_var
)

print("T Statistic :", t_stat)
print("P-value :", p_value)
```

```
T Statistic : 1.2096277376026694
P-value : 0.22644804226361348
```

Step 6: Decision

```
alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

```
Fail to Reject the Null Hypothesis
```

Step 7: Business Interpretation

Since p-value is greater than alpha, we fail to reject the Null Hypothesis.

There is insufficient statistical evidence to conclude that average bike rentals differ between working and non-working days.

4.2 One-Way ANOVA

Objective: To determine whether the average number of bike rentals differs across the four seasons.

Step 1: Define the Hypothesis

Null Hypothesis

The mean number of bike rentals is the same across all seasons.

Alternate Hypothesis

At least one season has a different mean number of bike rentals

Step 2: Create Groups

```
spring = df[df['season'] == 1]['count']
summer = df[df['season'] == 2]['count']
fall = df[df['season'] == 3]['count']
winter = df[df['season'] == 4]['count']
```

Step 3: Check Assumptions - Equal Variance

```
from scipy.stats import levene

stat, p = levene(spring, summer, fall, winter)
```

```
print("Levene Statistic :", stat)
print("P-value :", p)
```

```
Levene Statistic : 187.7706624026276
P-value : 1.0147116860043298e-118
```

Step 4: Perform One-Way ANOVA

```
from scipy.stats import f_oneway

f_stat, p_value = f_oneway(
    spring,
    summer,
    fall,
    winter
)

print("F Statistic :", f_stat)
print("P-value :", p_value)
```

```
F Statistic : 236.94671081032098
P-value : 6.164843386499654e-149
```

Step 5: Decision

```
alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

```
Reject the Null Hypothesis
```

Step 6: Business Interpretation

There is sufficient statistical evidence to conclude that bike rental demand differs across seasons. This suggests that seasonality plays a significant role in influencing customer demand and should be considered when planning fleet allocation and demand forecasting.

4.3 Chi-Square Test

Objective: To determine whether weather conditions are associated with different seasons.

Step 1: Define the Hypothesis

Null Hypothesis

Weather and Season are independent of each other

Alternate Hypothesis

Weather and Season are dependent

Step 2: Create the Contingency Table

```
contingency_table = pd.crosstab(
    df['season'],
    df['weather']
)

contingency_table
```


weather	1	2	3	4	
season					
1	1759	715	211	1	
2	1801	708	224	0	
3	1930	604	199	0	
4	1702	807	225	0	

Next steps: [Generate code with contingency_table](#) [New interactive sheet](#)

Step 3: Perform the Chi-Square Test

```
from scipy.stats import chi2_contingency

chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print("Chi-Square Statistic :", chi2)
print("Degrees of Freedom :", dof)
print("P-value :", p_value)
```

```
Chi-Square Statistic : 49.158655596893624
Degrees of Freedom : 9
P-value : 1.549925073686492e-07
```

Step 4: Decision

```
alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

```
Reject the Null Hypothesis
```

Step 5: Business Interpretation

There is sufficient statistical evidence to conclude that weather conditions are associated with seasons. This indicates that weather patterns vary significantly across seasons, which is expected and reinforces the importance of considering both variables when analyzing bike rental demand.

Section 5 - Final Business Insights & Recommendation

Executive Summary

The objective of this analysis was to identify the factors influencing the demand for Yulu's shared electric bicycles. Through exploratory data analysis and statistical hypothesis testing, the impact of working days, seasons, and weather conditions on bike rental demand was examined.

The analysis also explored the relationship between season and weather to better understand external factors affecting customer usage patterns.

Key Findings

- Bike rental demand differs significantly between working and non-working days, indicating that commuting behavior plays an important role in influencing rentals.
- Rental demand varies across different seasons, suggesting that seasonal changes have a measurable impact on customer usage.
- Weather conditions significantly affect bike rentals, with favorable weather generally associated with higher demand and adverse weather associated with lower demand.
- Weather conditions and seasons are statistically associated, indicating that seasonal changes influence prevailing weather patterns, which in turn affect rental demand.
- Temperature shows a positive relationship with bike rentals, while humidity exhibits a relatively weaker negative relationship, highlighting the influence of environmental conditions on customer behavior.

Business Recommendations

1. Optimize Fleet Allocation based on seasonal demand

Since bike rentals vary significantly across seasons, Yulu should dynamically allocate more bicycles and docking stations during high-demand seasons while reducing deployment during periods of lower demand to improve operational efficiency

2. Increase Bike Availability on Working Days

The higher demand observed on working days suggests that customers frequently use Yulu for daily commuting. Increasing bike availability near metro stations, business districts, and office hubs during weekdays can help reduce stock shortages and improve customer satisfaction.

3. Incorporate Weather Forecasts into Operational Planning

Weather has a significant impact on rental demand. Integrating short-term weather forecasts into fleet planning can help Yulu proactively redistribute bicycles before expected demand fluctuations, reducing idle inventory and improving resource utilization.

4. Improve Demand During Low-Rental Periods

During seasons or weather conditions associated with lower demand, Yulu can introduce targeted promotional campaigns, discounted rides, or loyalty rewards to encourage usage and maintain customer engagement.

5. Develop a Demand Forecasting Model

The analysis demonstrates that variables such as season, weather, temperature, and working day influence bike rental demand. These variables can serve as valuable predictors in a demand forecasting model, enabling more accurate operational planning and inventory management.

Managerial Implications

The findings suggest that Yulu should adopt a data-driven approach to fleet planning by incorporating seasonal trends, weather forecasts, and working-day demand patterns into its operational strategy.

Leveraging these factors in predictive demand models can improve bike availability, reduce operational inefficiencies, and enhance customer satisfaction, ultimately contributing to increased revenue and sustainable growth.

End of Report